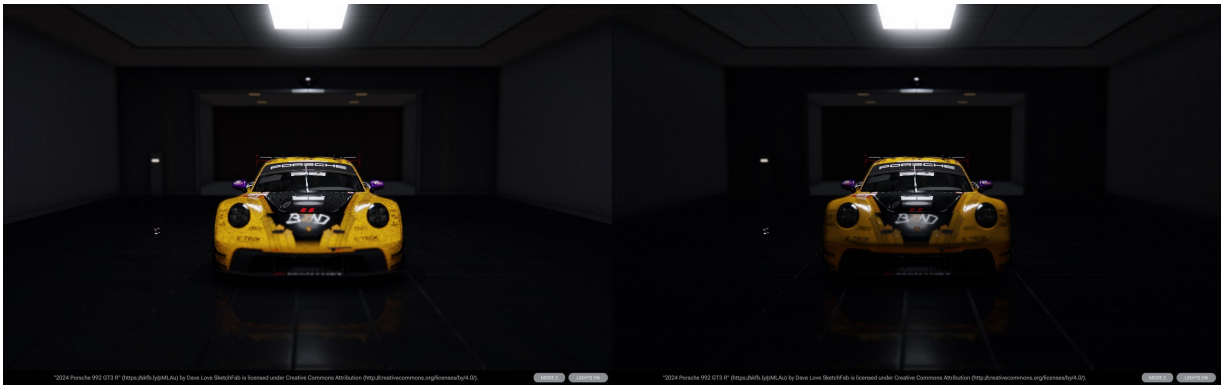
Texel emitters

Light through the holes in the black paper

A design document for the RageV renderer. It explains why a glowing texture used to light a room from the wrong place, what was done about it, and what the result measures.

The short version. A light fitting is usually a texture: a panel with the lit parts painted into it. The renderer treated the whole panel as glowing evenly, so a ceiling with four lit cells out of a hundred and forty-four lit the room as though all one hundred and forty-four were on. Thirty-six times too much light, arriving from the wrong place. This document is about teaching the renderer to aim at the four.



The same scene, the same frame, the same settings. Only the renderer differs: on the left the whole ceiling acts as a light, on the right the bounce is aimed at the four cells that are actually lit.

1 The problem

An emissive material makes a surface glow, and the traced bounce has to turn that glow into light on other surfaces. It did that by standing a **rectangle** in for the mesh and treating the whole rectangle as glowing evenly at the material's emissive value.

That is right for a panel that glows all over. It is wrong for a light *fitting*, where the emission is painted into a texture — four lit cells of a hundred and forty-four in the showroom's ceiling. The bounce then lit the room as though all hundred and forty-four were on: **thirty-six times too much light**, arriving from an eighty-six square metre phantom instead of from the four places the artist painted.

Its enormous sample-to-sample variance is what showed up as grain crawling over the car's paint with the camera perfectly still.

Before. One textured mesh for the ceiling. The whole panel acts as a light, so the room is flooded and the paint crawls with grain that never settles.

Why it was hard to see

The ceiling *looked* right. Only what it emitted into the room was wrong, and nothing on screen said so. Worse, the workaround — splitting the lit cells out into their own mesh — made the rectangle true again, so the picture came good for a reason that had nothing to do with the code. Two wrong diagnoses came first.

2 The idea

> A pixel shader runs per pixel, so a pixel whose emissive value clears a threshold could be counted as a source of light — a directional light behind black paper with holes, where light escapes only through the holes.

The research placed that in the technique landscape. It is **textured-light importance sampling**, it has an established literature, and the **texture domain is its correct home**: screen space is view-dependent by construction — a hole that leaves the frame stops existing as a light — and this engine's screen-space GI already does that half. Texels are known ahead of time, from any camera. It is what offline renderers do for textured area lights, and it scales down to real time almost unchanged: the tables build at load, and the shader pays a couple of fetches per sample.

It also fitted the shape the engine already had. Analytic lights travel one road; emissive surfaces travel another. The second road was the one assuming a mesh glows uniformly, and it is the only road this work touches.

3 The implementation

Three stages, each shippable and testable on its own.

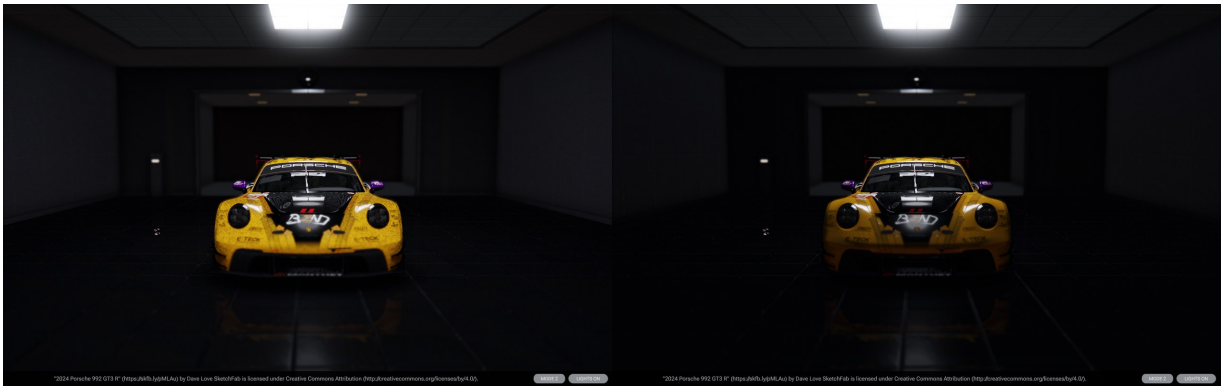
Stage	What it does	Why it was needed
0	The bounce subtracts a surface's emissive only where the emitter list actually answers for it.	It subtracted whenever the list was non-empty *at all*, though membership is filtered three ways — so light from anything the list left out was removed by one estimator and added by none. It simply vanished.
1	An emissive map's **mean**, read at load, folded into the emitter's radiance.	Makes the *total power* right for any map, in any scene, however authored. The phantom becomes impossible.
2	A **32×32 luminance table** per map. The sampler picks a cell in proportion to what it emits, maps it back through an affine uv map, and reads the real texel.	Makes the light arrive from *where it is painted*, not spread over the whole rectangle.

What it will and will not do

Stage 2 engages for the flat primitives whose texture coordinates the engine can state exactly, with untransformed uv, and a map whose distribution is worth aiming at. Anything else — a modelled fitting, a tiled material, a uniform map — falls back to stage 1, **which is never wrong, only average**. That fallback is the whole safety argument: the worst case is the behaviour the engine had before, not a new failure.

4 What it measures

Same scene file, same frame, same settings. Only the binary differs.



Left, before. The whole ceiling acts as a light. **Right, after.** The same scene and the same frame, with the bounce aimed at the lit texels. The two differ over 1.5 million pixels, by up to 175 levels.

	grain on the paint	car	wall	floor	frame-to-frame crawl
before	3.31	64.5	19	17	91 levels
after	**2.31**	**35.9**	**11**	**11**	**11 levels**
the hand-split workaround, for reference	2.34	34.8	—	—	7 levels

The last row is the point. The hand-split version is what the scene looked like after an artist pulled the lit cells out into their own mesh by hand — surgery on the scene to work around a defect in the renderer. The feature reaches the same picture from plain authoring, and spends no emitter slots doing it.

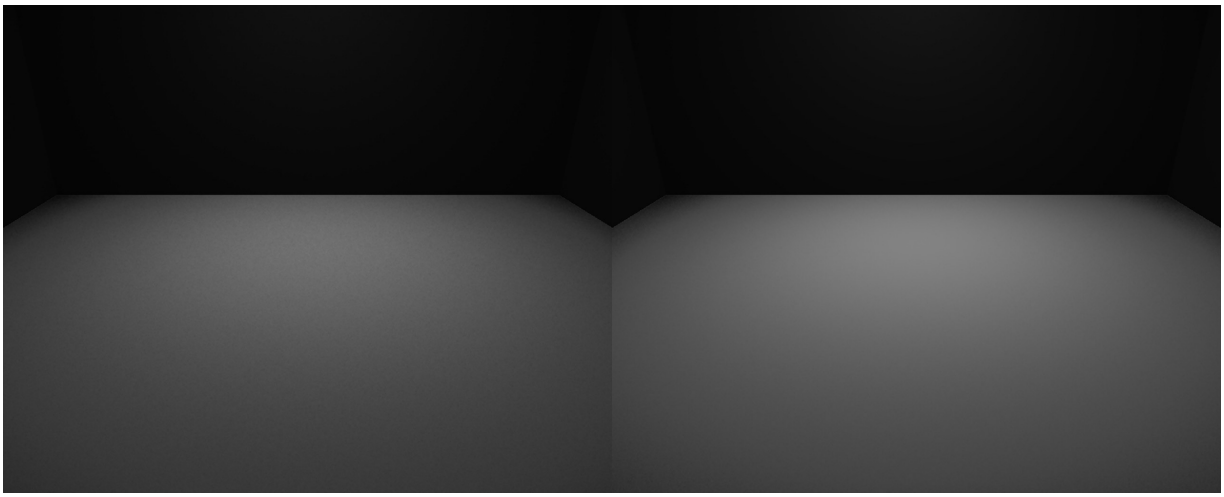
The workaround this replaces. Correct, but it cost a second mesh, a second material, and an artist's afternoon — per fitting.

On a purpose-built fixture

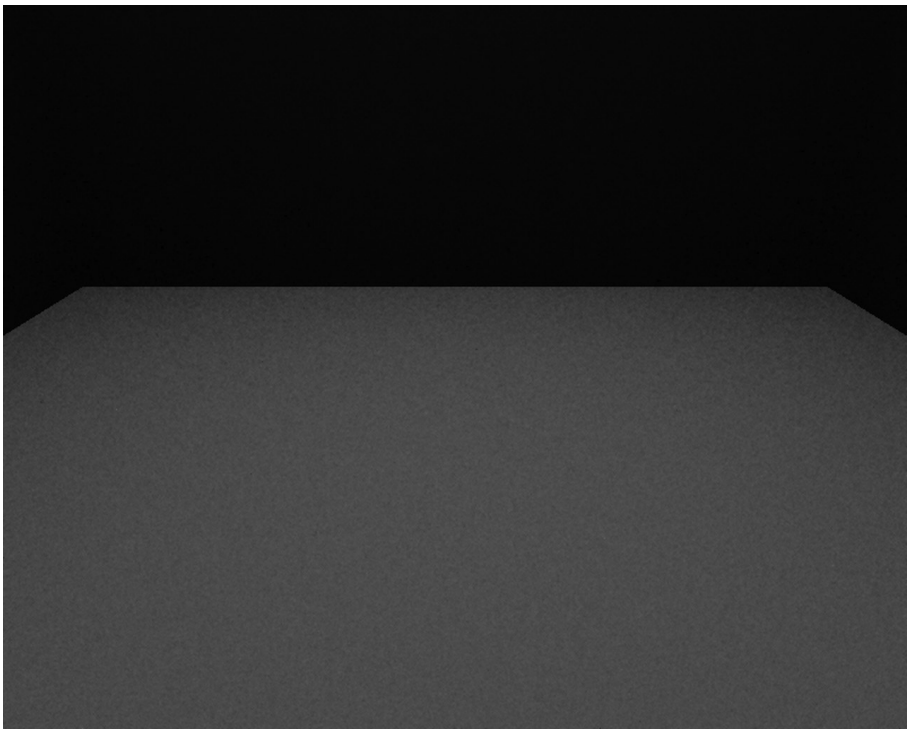
A dark room, one ceiling, four lit cells of a hundred and forty-four. The middle row is the ground truth: the same four cells built as four separate emitters, which the estimator represents exactly.

	floor brightness	far-to-near gradient
textured mesh, aimed	**82.53**	**1.40**
four separate emitters (ground truth)	82.54	1.41
the same total power spread evenly	74.23	0.99

The gradient is the claim that only aiming can satisfy. Spread the same power evenly over the ceiling and the floor lights flatly — 0.99, no gradient at all. Aim it, and the light falls off from the fittings the way it does when the fittings are really there.



Left: one textured mesh, aimed. **Right:** the same four cells as four separate emitters — the ground truth. 82.53 against 82.54, and 1.40 against 1.41.



And the same total power spread evenly, which is what the renderer did before: the floor lights flatly, gradient 0.99.

5 How it is guarded

Four claims, and **every one of them was watched to fail** before it was trusted — a threshold nobody has seen go red is a threshold nobody has read.

The claim	What breaking it does
A partly-lit surface emits its own power, not its whole rectangle's.	Revert the fold: 3.33× too bright.
An emitter over there does not change a glow over here.	Revert the flag: the room goes **exactly black** — a sealed emitter in a corner deleting every photon in the room.
The cooked and the raw texture paths agree on the map's mean.	Revert the mip choice: 1.73× apart.
The light arrives from where it is painted.	Disable the aiming: the gradient collapses from 1.40 to 1.00.

6 What it costs

Nine interleaved pairs in both orders measured **7.13 ms before and 7.50 ms after**, about five per cent. That number should be read with care, and the reasons are worth stating plainly rather than burying.

The traced GI pass itself did not get slower, and a pass-by-pass comparison put the two builds within 0.05 ms — so the five per cent never landed on any nameable pass. A later investigation went looking for it and **could not find a mechanism**: every candidate path, costed from the code, came to single-digit microseconds against the three hundred and seventy the gap needed.

Two things undermine the measurement itself. The comparison set a smoothed running average for the pass against a true mean for the frame, which are different statistics and not differenceable. And the scene it was taken on is not the one that ships — it had to be rebuilt by hand for the test and was not kept.

The honest position: an unexplained and unreproduced five per cent, on a build that is also doing more correct work than the one it is compared against. It is recorded here because it was measured, not because it is understood.

7 The mistakes worth retelling

A careless revert filter cost the shader fix three times. A pattern meant to match one family of files also matched the trace shader. Twice the fix was committed as dead code and the checks still passed — because the runtime compiles the shaders staged beside the executable, and those still held it.

A cooked texture's smallest mip is not its mean. The cooker's box filter drops an odd level's tail row, so walking all the way down discards five texels of nine: 2.27× wrong, and a shifted variant reads exactly zero. Walk to the deepest evenly-halved level instead.

A check that tests the path nothing ships is not a check. The raw texture path was exact while the cooked one — the one that ships — was 2.27× wrong. The check was only looking at the raw one.

The split-mesh room is not a ground truth for total power. Equal power in a different place delivers a different fraction of itself to the floor. It is the right reference for *where* the light lands, and the wrong one for *how much*.

A plane's v runs opposite to an image's rows, and the two cancel. Get it wrong and the light goes to the opposite half of the room, showing up in no total anywhere. Count cells from the corner.

Full engineering record, every measurement and the rejected alternatives: **docs/TEXEL-EMITTERS.md**. Guarded by **tools/scripts/check_emitters.py** over the fixtures **make_emitter_scene.py** writes.